❺ Multiplier to compute the next interval

The retry behavior is enabled only when the `spring.cloud.config.fail-fast` property is set to `true`. You probably don't want to retry in your local environment if the config server is down, especially considering we made it an optional backing service. Feel free to test the application's behavior when retrying the connection with the config server down, but remember to set the `fail-fast` property back to `false` if you want to keep it optional in your local environment. In production you can set it to `true` using one of the strategies covered in this chapter. When you're done testing the applications, stop both of them with Ctrl-C.

You are now ready to use the Config Service to configure any application you want. However, there's still an aspect I haven't covered yet. How can we change the configuration at runtime?

4.4.3 Refreshing configuration at runtime

What happens when new changes are pushed to the Git repository that's backing the Config Service? For a standard Spring Boot application, you would have to restart it when you change a property (either in a property file or an environment variable). However, Spring Cloud Config gives you the possibility to refresh configuration in client applications at runtime. Whenever a new change is pushed to the configuration repository, you can signal all the applications integrated with the config server, and they will reload the parts affected by the configuration change. Spring Cloud Config offers different options for doing that.

In this section, I'll show you a simple refresh option consisting of sending a special `POST` request to a running Catalog Service instance to trigger the